



RIF SAS

Rassemblement des Ingénieurs Francophones

INFORMATIONS DU CANDIDAT

Nom & Prénom	_____
École / Université	_____
Évaluateur	_____
Date du test	_____
Heure de début	_____

Document confidentiel — usage interne RIF SAS — ne pas diffuser

1. Contexte & Objectif

Ce test de sélection évalue votre capacité à déployer une application professionnelle en environnement conteneurisé. Vous disposez de 8 heures pour réaliser les 3 parties. Tout tourne sur votre machine locale — aucun serveur distant, aucun compte cloud requis.

L'évaluateur récupère votre dépôt Git en fin de journée et rejoue vos livrables de façon autonome. La notation est donc entièrement objective.

Ce que ce test évalue

Docker & Docker Compose	Savoir écrire un docker-compose.yml fonctionnel multi-services.

Réseaux & volumes	Isoler les services, assurer la persistance des données.
Sécurité basique	Gérer les secrets correctement, ne jamais commiter un .env.
Automatisation Bash	Écrire un script de sauvegarde simple et reproductible.
Git & documentation	Commits propres, README lisible, livrables organisés.

2. Règles du test

2.1 Utilisation de l'IA

 Autorisé	ChatGPT, Claude, Copilot pour générer du code ou débloquer un problème technique.
 Obligatoire	Tenir un fichier docs/journal-ia.md : noter les 3 prompts les plus utilisés + ce que vous en avez retenu.
 Interdit	Copier-coller du code que vous ne pouvez pas expliquer lors du débrief oral.

2.2 Git

- Un dépôt Git personnel (GitHub ou GitLab.com), accessible à l'évaluateur en fin de journée.
- Commits réguliers tout au long de la journée — pas un seul commit géant à 17h.
- Aucun secret en clair (.env, mot de passe, clé) — malus -10 pts immédiat.

2.3 Débrief oral (15 min en fin de journée)

L'évaluateur vous posera 3 questions sur votre code. L'objectif n'est pas de piéger, mais de vérifier que vous comprenez ce que vous avez produit. Soyez honnête : dire « j'ai utilisé l'IA pour cette partie et voici ce que j'en ai compris » est bien vu.

3. Prérequis machine

OS	Ubuntu 20.04+ ou WSL2	uname -a
Docker Engine	v24+	docker --version
Docker Compose	v2+	docker compose version
Git	v2+	git --version
RAM disponible	4 Go minimum	free -h
Disque libre	5 Go minimum	df -h
Internet	Requis le matin (pull images)	docker pull odoo:17

⚠️ Prévoir 30 min la veille pour vérifier ces prérequis et pré-télécharger les images Docker si votre connexion est lente.

Stack Odoo conteneurisée · 35 pts

Contexte

RIF SAS déploie Odoo ERP pour ses clients. Votre mission : assembler la stack Odoo + PostgreSQL en Docker Compose, gérer les secrets correctement et valider que tout fonctionne.

Tâche 1.1 — Écrire le docker-compose.yml · 15 pts

Ce qu'on attend

- Déclarer deux services : odoo (image odoo:17) et db (image postgres:15).
- PostgreSQL isolé dans un réseau Docker privé — non exposé directement sur l'hôte.
- Odoo exposé sur le port 8069 de la machine locale.
- Tous les secrets (mot de passe DB, nom de base) dans un fichier .env non commité.
- Fournir un .env.example documenté avec les clés attendues.

Grille — Tâche 1.1

Stack démarre sans erreur	docker compose ps : les 2 services Up	5
Odoo accessible sur http://localhost:8069	Page de login Odoo visible dans le navigateur	4
PostgreSQL isolé dans réseau privé	docker inspect db : aucun port publié	3
.env absent du dépôt, .env.example présent	git log -- .env retourne vide	3

Tâche 1.2 — Persistance des données · 10 pts

Ce qu'on attend

- Configurer des volumes Docker nommés pour PostgreSQL (postgres-data) et Odoo (odoo-filestore).
- Créer une base de démonstration Odoo et installer le module Ventes.
- Prouver la persistance : docker compose down puis docker compose up -d — le module Ventes doit être encore présent.

Grille — Tâche 1.2

Volumes nommés déclarés dans docker compose.yml	docker volume ls montre les 2 volumes	3
Module Ventes installé avant down	Capture écran Odoo avec module visible	3

Données persistantes après down/up	Module Ventes encore présent après redémarrage	4
------------------------------------	--	---

Tâche 1.3 — Reverse proxy Nginx · 10 pts

Ce qu'on attend

- Ajouter un 3e service nginx dans le docker-compose.yml.
- Nginx proxyfie http://localhost:80 vers le conteneur Odoo (port 8069).
- Ajouter l'entrée 127.0.0.1 erp.local dans /etc/hosts et accéder à Odoo via http://erp.local.
- La configuration Nginx doit être dans un fichier versionné (apps/nginx/odoo.conf).

Grille — Tâche 1.3

Service nginx ajouté et démarré	docker compose ps : 3 services Up	3
http://erp.local charge Odoo	Capture navigateur sur erp.local	4
Config Nginx versionnée dans Git	Fichier apps/nginx/odoo.conf présent	3

✓ Checkpoint 12h00 — Livrables requis

- docker-compose.yml fonctionnel avec les 3 services.
- .env.example avec toutes les variables documentées.
- Capture : Odoo accessible via http://erp.local.
- Capture : module Ventes présent après docker compose down/up.

Sauvegarde & Automatisation · 35 pts

Contexte

Un système en production doit pouvoir être sauvegardé et restauré. Votre mission : écrire un script de backup complet et valider un scénario de perte totale des données.

Tâche 2.1 — Script de sauvegarde · 15 pts

Ce qu'on attend

- Écrire un script Bash apps/backup.sh qui réalise :
- → pg_dump de la base Odoo via docker exec (sans arrêter les conteneurs). • → Archive tar.gz du filestore Odoo.
- → Nom de fichier horodaté : backup_YYYYMMDD_HHMMSS.tar.gz. • → Log de chaque opération dans /var/log/backup.log.
- Rendre le script exécutable (chmod +x) et le tester une première fois. • Ajouter une entrée cron pour exécuter le backup chaque nuit à 02h00.

Grille — Tâche 2.1

Script créé, exécutable, syntaxe valide	bash -n backup.sh passe sans erreur	3
pg_dump réussi sans arrêter les conteneurs	Fichier .sql présent dans l'archive	4
Archive tar.gz horodatée créée	ls /backup/ montre le fichier avec date	4
Entrée cron configurée à 02h00	crontab -l montre 0 2 * * * ...	4

Tâche 2.2 — Simulation de crash & restauration · 20 pts

Scénario

Vous simulez la perte totale du système : suppression des conteneurs ET des volumes. Vous devez ensuite tout restaurer depuis votre backup et prouver qu'Odoo fonctionne à nouveau.

1	Exécuter backup.sh	Archive créée dans /backup/
2	docker compose down -v (supprime les volumes)	Stack détruite, données perdues
3	Restaurer le dump via docker exec + psql	Base PostgreSQL recrée
4	Restaurer le filestore depuis l'archive tar.gz	Volume Odoo restauré

5	docker compose up -d	☒ Odoo opérationnel, module Ventes présent
---	----------------------	--

Grille — Tâche 2.2

Crash simulé correctement (volumes supprimés)	docker volume ls vide avant	3

	restauration	
Base PostgreSQL restaurée depuis le dump	Odoo se connecte à la DB après up	7
Odoo fonctionnel après restauration complète	Module Ventes visible, données intactes	7
Runbook de restauration écrit dans docs/	docs/restauration.md lisible par un tiers	3

✓ Checkpoint 16h00 — Livrables requis

- apps/backup.sh exécutable et testé.
- Capture de chaque étape du scénario de crash.
- Odoo fonctionnel après restauration (capture écran).
- docs/restauration.md écrit.

Documentation & Livrables finaux · 30 pts

Contexte

Un bon DevOps ne livre pas seulement du code qui fonctionne — il livre du code que les autres peuvent comprendre et maintenir. Cette partie évalue la qualité de vos livrables.

Tâche 3.1 — README.md · 10 pts

Ce qu'on attend

- Un README.md à la racine du dépôt qui explique :
- → Comment démarrer la stack (de zéro) en moins de 5 commandes.
- → Les prérequis machine.
- → Comment lancer une sauvegarde et restaurer.
- L'évaluateur doit pouvoir rejouer tout le projet sans vous demander d'aide.

Grille — Tâche 3.1

Démarrage en 5 commandes maximum	README clair, commandes copy pastables	5
Section backup & restauration présente	Instructions reproductibles par un tiers	5

Tâche 3.2 — Qualité Git · 10 pts

Ce qu'on attend

- Au moins 5 commits répartis tout au long de la journée (pas un seul commit final).
- Messages de commits clairs et descriptifs (ex : feat: add nginx reverse proxy).
- .gitignore correct : .env, *.log, __pycache__ exclus.
- Arborescence propre et organisée.

Grille — Tâche 3.2

5+ commits répartis dans la journée	git log --oneline montre l'historique	4
Messages de commits descriptifs	Lecture du git log est parlante	3
.gitignore correct, aucun secret commité	git ls-files ne montre pas .env	3

Tâche 3.3 — Journal IA · 10 pts

Ce qu'on attend

- Un fichier docs/journal-ia.md avec :
 - → Les 3 prompts les plus importants que vous avez utilisés.
 - → Pour chaque prompt : ce que l'IA a généré + ce que vous avez modifié + pourquoi.
- → Une phrase sur ce que vous avez appris de nouveau aujourd'hui.

Grille — Tâche 3.3

3 prompts documentés avec contexte	Fichier présent et complet	5
Recul critique sur l'utilisation de l'IA	Modifications et apprentissages expliqués	5

✓ Livraison finale 17h00 — Livrables requis

- Lien du dépôt Git transmis à l'évaluateur avant 17h00.
- README.md complet à la racine.
- docs/restauration.md.
- docs/journal-ia.md.
- Présence au débrief oral de 15 min.

4. Scoring global — 100 points

P1	Stack Odoo conteneurisée (Docker Compose, volumes, proxy)	35 pts
P2	Sauvegarde & restauration après crash	35 pts

P3	Documentation, qualité Git & journal IA	30 pts

Malus

Secret (.env, mot de passe) commité sur Git	-10 pts immédiat
Code non expliqué lors du débrief oral	-5 pts / occurrence
Dépôt Git transmis après 17h00	-5 pts

Barème de sélection

85 – 100	 Sélectionné	Autonome, rigoureux, bonne maîtrise Docker. Prêt pour le bootcamp 3 jours.
65 – 84	  À évaluer	Bases solides mais lacunes sur la persistance ou la restauration. Débrief oral déterminant.
< 65	 Non retenu	Niveau insuffisant pour intégrer le programme DevOps.

5. Arborescence attendue du dépôt Git

test-selection-devops/

```

├── apps/
│   ├── docker-compose.yml # Stack complète
│   ├── .env.example # .env JAMAIS commité
│   ├── backup.sh # Script sauvegarde
│   └── nginx/
│       └── odoo.conf # Config reverse proxy
├── docs/
│   ├── restauration.md # Runbook PRA
│   └── journal-ia.md # Journal prompts IA
├── .gitignore
└── README.md # Guide de démarrage

```